Sample MT1 Lab Examination

Shopping Cart Management Web Application

Full Stack Development – Node.js Lab Examination

Technology: Node.js, HTTP Module, FS Module, Express.js
Database: Not Required (Use JavaScript Array)

---

1. Problem Statement

Develop a **Shopping Cart Management Web Application** using Node.js and Express.js.

The application must demonstrate the working of the Node.js HTTP module, routing, handling different HTTP methods, serving static files using the FS module and implementing REST APIs using the Express.js framework.

You are required to implement the application in separate modules as described below.

2. Project Setup

Create a project directory named:

`MT1-LABEXAM`

Initialize the Node.js project using ES Modules and install Express.js using npm.

GitHub Instructions:

- Start the lab examination by creating a GitHub repository named `MT1-LABEXAM`.
- Push the initial project setup with an initial commit.
- Update the repository after completing each module.
- Use meaningful commit messages for every update.

3. Expected Project Structure

Organize your project using the following directory structure:

```
ShoppingCart/
│
├── package.json
│
├── httpServer.js
│
├── expressApp.js
│
├── public/
```

```
│   └── index.html
│
└── screenshots/
```

4. Implementation Requirements

Part A: HTTP Module and Server Creation

Implement the following functionalities in `httpServer.js`.

A1. Create an HTTP Server

- Import the built-in `http` module.
- Create an HTTP server using `http.createServer()`.
- Configure the server to listen on port `3000`.
- Display an appropriate message when the server starts.

A2. Implement Routing

Implement the following routes:

Route	Expected Response
`/`	Welcome message in HTML format
`/cart`	List of shopping cart items in JSON format
`/about`	Shopping cart application information
`/home`	Serve the HTML page using the FS module

A3. Handle HTTP Methods

Implement the following operations:

GET /cart

- Retrieve and display all shopping cart items.
- Return the response in JSON format.

POST /add-to-cart

- Accept product details in JSON format.
- Required fields: `id`, `productName`, `price` and `quantity`.
- Add the new product to an array.
- Return an appropriate success message.

A4. Static File Serving Using FS Module

- Import the built-in `fs` module.
- Read the `index.html` file from the `public` directory.
- Serve the HTML file when the user accesses `/home`.
- Set the appropriate Content-Type header.

- Handle file-reading errors appropriately.

A5. Error Handling

- Return HTTP status code `404` for invalid routes.
- Set appropriate Content-Type headers.
- Handle invalid requests appropriately.
- Validate the required product details before adding an item.

Part B: Shopping Cart REST API Using Express.js

Implement the Express.js application in `expressApp.js`.

B1. Configure Express.js

- Import Express.js.
- Create an Express application.
- Configure middleware to parse incoming JSON requests using `express.json()`.
- Initialize an array containing at least three sample shopping cart items.
- Configure the application to listen on port `4000`.

B2. Implement CRUD Operations

Implement the following REST API endpoints:

HTTP Method	Endpoint	Functionality
GET	`/cart`	Retrieve all cart items
GET	`/cart/:id`	Retrieve a cart item by ID
POST	`/cart`	Add a new product to the cart
PUT	`/cart/:id`	Update product details
DELETE	`/cart/:id`	Remove a product from the cart

B3. Shopping Cart Object Structure

Each shopping cart item should contain the following properties:

```
{
  "id": 1,
  "productName": "Wireless Mouse",
  "price": 799,
  "quantity": 2,
  "category": "Electronics"
}
```

Initialize at least three sample products in the shopping cart array.

B4. Additional Requirements

- Use route parameters to identify individual cart items.
- Accept product information through JSON request bodies.
- Return appropriate JSON responses.
- Return HTTP status code `201` when a product is successfully added.
- Return HTTP status code `404` when a product is not found.
- Handle invalid requests appropriately.
- Calculate the total price of an individual cart item using `price × quantity`.
- Maintain the updated shopping cart in the JavaScript array.

---

5. Running the Application

Step 1: Start the HTTP Server

Open the terminal and execute:

```
node httpServer.js
```

Expected output:

```
HTTP Server running at http://localhost:3000
```

Open the following URLs in your browser:

```
http://localhost:3000/
http://localhost:3000/cart
http://localhost:3000/about
http://localhost:3000/home
```

Verify that the HTML page is successfully served using the FS module.

Step 2: Start the Express Application

Open another terminal and execute:

```
node expressApp.js
```

Expected output:

```
Express Server running at http://localhost:4000
```

6. API Testing Using Postman

Test all the following endpoints using Postman.

Method	URL	Expected Result
GET	http://localhost:4000/cart	Display all cart items
GET	http://localhost:4000/cart/1	Display cart item with ID 1
POST	http://localhost:4000/cart	Add a new product
PUT	http://localhost:4000/cart/1	Update product details
DELETE	http://localhost:4000/cart/1	Delete product from cart

Sample POST Request

Select POST in Postman.

URL:

```
http://localhost:4000/cart
```

Select **Body → raw → JSON**.

Request Body:

```
{
  "id": 4,
  "productName": "Bluetooth Headphones",
  "price": 1499,
  "quantity": 1,
  "category": "Electronics"
}
```

Expected Response:

```
{
  "message": "Product added successfully",
  "product": {
    "id": 4,
    "productName": "Bluetooth Headphones",
    "price": 1499,
    "quantity": 1,
    "category": "Electronics"
```

```
  }
}
```

Sample PUT Request

Select PUT in Postman.

URL:

```
http://localhost:4000/cart/1
```

Request Body:

```
{
  "productName": "Wireless Mouse",
  "price": 899,
  "quantity": 3,
  "category": "Electronics"
}
```

The application should update the existing product details and return an appropriate success response.

Sample DELETE Request

Select DELETE in Postman.

URL:

```
http://localhost:4000/cart/1
```

The application should remove the product with ID 1 and return an appropriate success message.

---

7. General Guidelines

1. Use Node.js and JavaScript (ES6 syntax).
2. Use the built-in HTTP and FS modules wherever specified.
3. Do not use Express.js for Part A.
4. Use Express.js for Part B only.
5. Do not use any database. Maintain shopping cart records in an array of objects.
6. Both applications must run independently on their specified ports.
7. Use meaningful variable names and proper indentation.
8. Ensure that all routes work correctly.
9. Handle invalid routes, missing products and file errors.
10. Do not copy another student's implementation.

11. Ensure that the application works correctly with Postman.
12. Use proper GitHub commits after completing each module.

---

8. Submission Requirements

Students must submit:

1. Push the complete project to GitHub with proper commit messages. Repository name must be `MT1-LABEXAM`.
2. `package.json` file.
3. Screenshots of the HTTP server running successfully.
4. Screenshots demonstrating static file serving using the FS module.
5. Postman screenshots demonstrating GET, POST, PUT and DELETE operations.
6. A properly organized project folder.
7. Screenshots of the GitHub repository showing the commit history.

Important: The application must be demonstrated successfully to the examiner. Marks will be awarded based on implementation, functional correctness, error handling, code quality, GitHub commit history and practical demonstration.